SERVER:
COMPOSABLE MEMORY
SYSTEMS (CMS)



# Managing Orchestration Memory with the CMS Memory Fabric Manager


Hongjian Fan *Sr Staff Engineer, Seagate Technology*

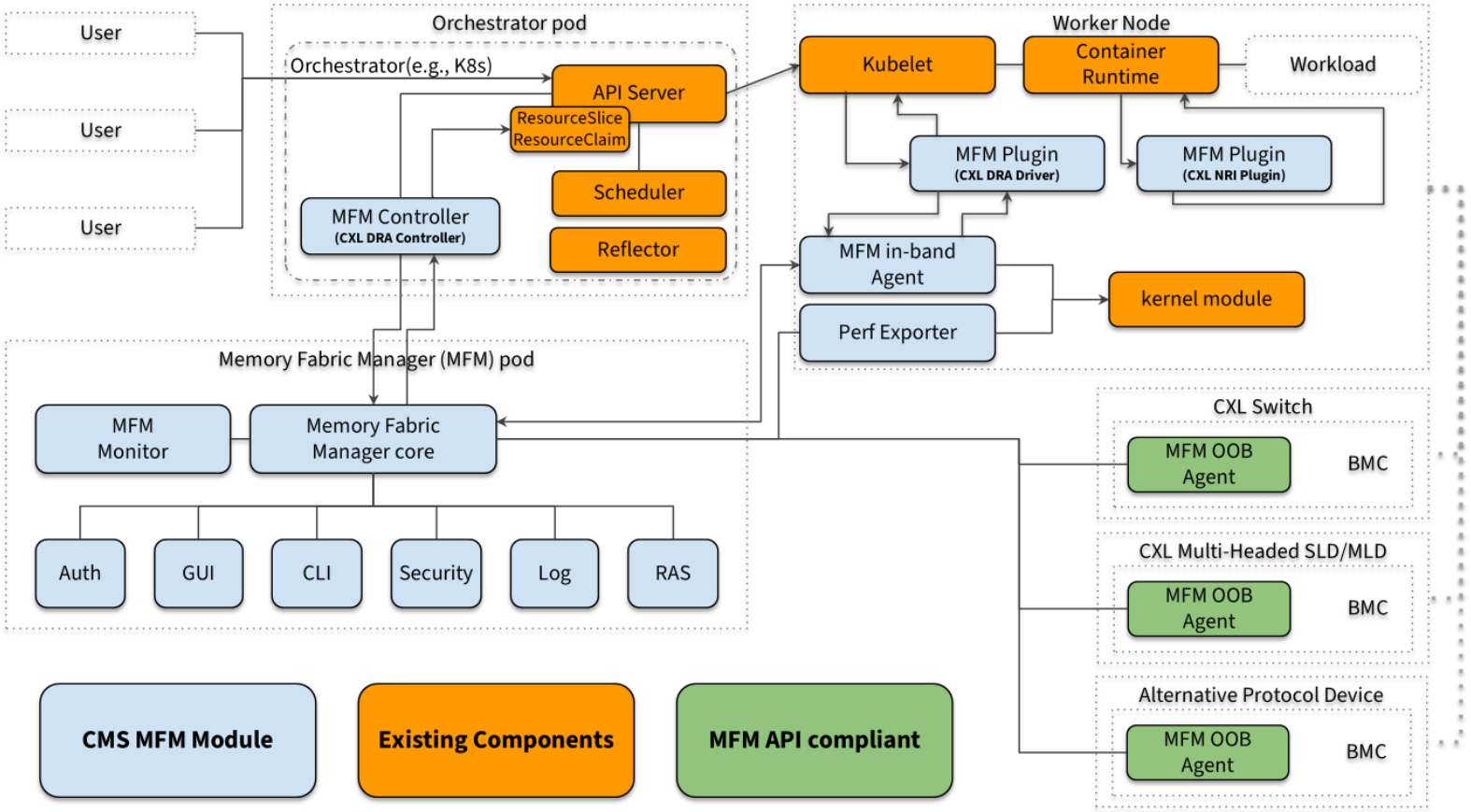
Han Deok Lee *Principal Engineer, Samsung*


OCP PLATINUM Member

# CMS memory fabric Orchestration workstream

- Workstream Goal:
  - Identify fabric management requirements for composable memory systems, develop fabric management software, and collaborate with industry standards bodies to create open, interoperable management schemas and reference implementations.
- Current Focus:
  - Continuously contribute to the Fabric Orchestration software definition
  - CXL 3.x/4.x, scale up network, alternative memory protocol support
- Potential Contributions
  - OCP CMS Memory Fabric Orchestration Architecture White Paper Revision 1.1
  - Memory Fabric Orchestration Software Requirement – work in progress
  - Software contribution:
    - Samsung – Pangaea -- Samsung Composable Memory System (CMS)

# Management Software




Modular Components:

- MFM pod
  - Management core
  - Monitor
  - Auth/Log
  - Security/RAS
  - GUI/CLI
- Worker Node
  - MFM in-band agent
  - DRA plugin / NRI plugin
  - Perf Exporter
- Control Node
  - MFM DRA controller

# Two forms of CXL memory

CXL Type-3 memory is commonly exposed in **two modes**:

1. **System RAM mode (CPU-less NUMA node)**: kernel-managed memory tier.
2. **DAX (Direct Access) mode (/dev/daxX.Y)**: application-managed memory region

## Compare NUMA node to DAX device

| Aspect | CXL as System RAM (NUMA Node) | CXL as DAX Device (devdax) |
|---|---|---|
| Linux representation | CPU-less NUMA node | /dev/daxX.Y character device |
| Memory owner | Linux kernel page allocator | User application |
| Access method | malloc(), anonymous mmap(), NUMA policies | File-backed mmap() on DAX device |
| Application changes | Minimal or none | Requires DAX-aware code |
| Memory tiering | AutoNUMA, memory tiering, page migration | No automatic tiering |
| Sharing between processes | Standard shared memory mechanisms | Explicit shared-memory mappings possible |
| Sharing between hosts | Difficult and potentially unsafe | Well-suited for shared memory architectures |
| Risk of kernel consuming memory | Yes | No |
| Container orchestration | Harder to allocate exact regions | Easy to expose specific DAX regions |
| Dynamic provisioning | Limited | Better fit for fabric memory managers |
| Best use case | Memory expansion, transparent capacity growth | Shared memory pools, KV-cache offload and transfer |

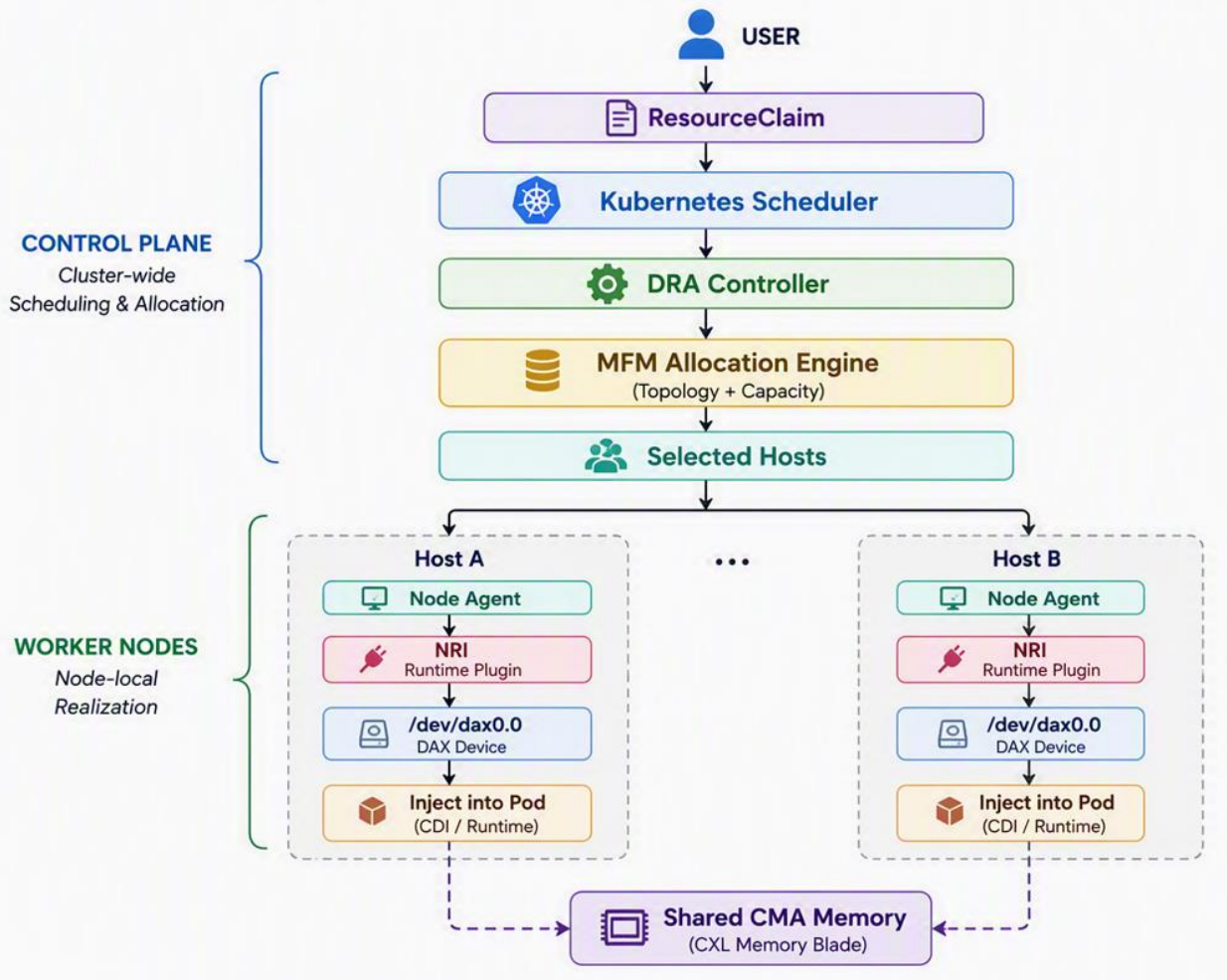
# Orchestration Integration: DRA+NRI

**DRA ( Dynamic Resource Allocator )**

- Global resource discovery
- Capacity tracking
- Fabric attached device selection
- Host selection
- Allocation policy
- Scheduler integration

**NRI ( Node Resource Interface )**

- Local DAX creation
- Device binding
- Runtime injection
- Memory online/offline
- Container configuration



# Classifying the Use Cases

- Each use case sits on two axes: operation scope (pod-level vs. node-level) and host-side memory representation (System-RAM vs. devdax).

- **Node-level** cases are driven by the Kubelet MFM Plugin (DRA Driver, §5.3.1).
- **Pod-level** cases are driven by the Container Runtime MFM Plugin (NRI Plugin, §5.3.2).
- **MFM in-band Agent (§5.4)** performs the underlying host-side memory operation in every case.

***Asymmetry to note:*** *the two pod-level cases (§8.2, §8.4) differ in how often the NRI Plugin participates — see next slide.*

| | System-RAM | devdax |
|---|---|---|
| **Node-level** | **8.1**<br>Node-Level Memory Allocation via System-RAM (NUMA Node) | **8.3**<br>Node-Level Memory Allocation via devdax (Character Device) |
| **Pod-level** | **8.2**<br>Pod-Level Online Memory Resizing via System-RAM | **8.4**<br>Pod-Level Dynamic devdax Unit Allocation |

# Why the Pod-Level Cases Aren't Symmetric

- Both §8.2 and §8.4 add/remove capacity from a running pod without a restart — but the Container Runtime MFM Plugin (NRI Plugin) participates very differently in each.

### 8.2

**System-RAM Online Resizing**

- NRI Plugin is invoked on every resize, via the UpdatePodSandbox event.
- Standard path exists: cgroup /resize subresource + NRI UpdatePodSandbox.

### 8.4

**devdax Dynamic Unit Allocation**

- NRI Plugin is invoked only once — at initial CreateContainer — to pre-configure device-cgroup permissions and mount propagation.
- Every later add/release is driven directly by the MFM in-band Agent's own pressure-triggered loop (“Auto Injection”), no further NRI round-trip.

**Root cause:** Kubernetes and mainstream container runtimes provide a standard in-place resize path for cgroup-based System-RAM, but no equivalent mechanism exists to re-apply a CDI Spec to an already-running container.

# Node-Level Memory Allocation via System-RAM

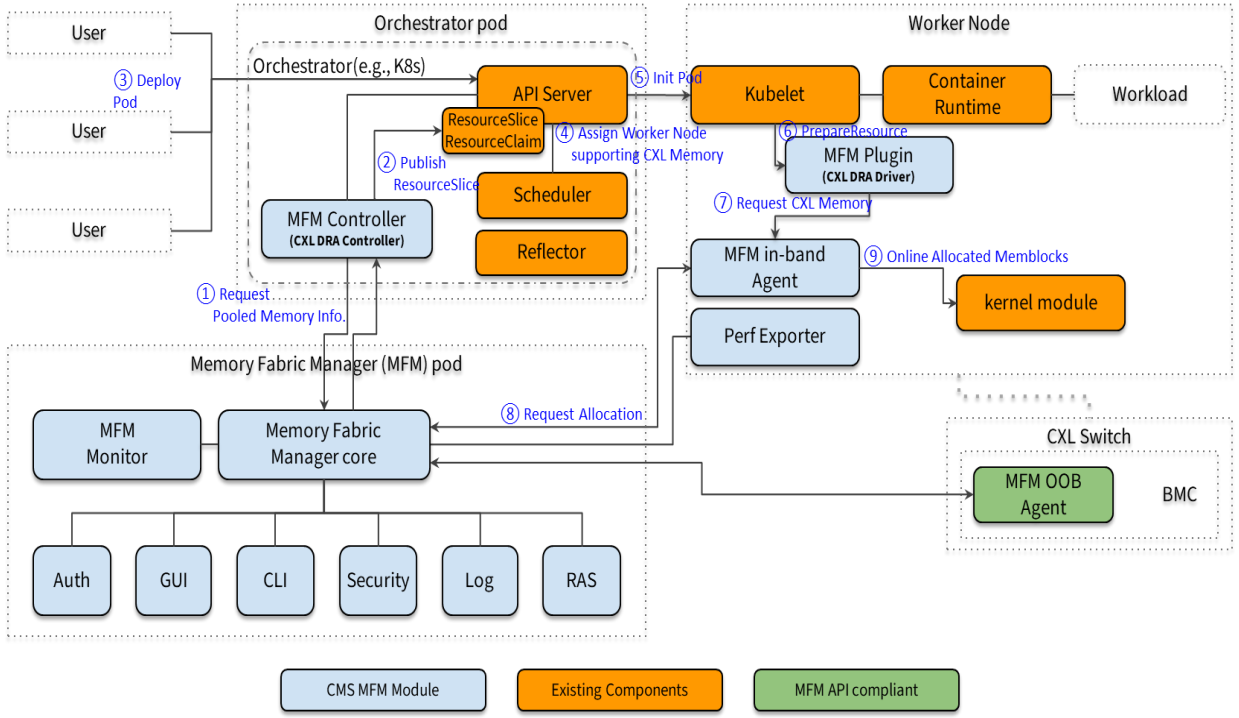
8.1

**SCENARIO**

A new host node joins the cluster (or an existing node's fabric-attached memory changes). The composed memory is exposed as a single CPU-less NUMA node backed by System-RAM memory blocks, available to any pod scheduled on that node.

**KEY POINTS**

- Baseline case
  : Kubernetes itself, not tied to any single application's behavior.
- Goal
  : make CXL capacity fungible with local DRAM at whole-node granularity.
- Ordinary workloads consume it via standard interface,
  with no device awareness.
- Driven by the Kubelet MFM Plugin's NodePrepareResources path (DRA), coordinated by the MFM Controller (§5.2).



# Pod-Level Online Memory Resizing via System-RAM

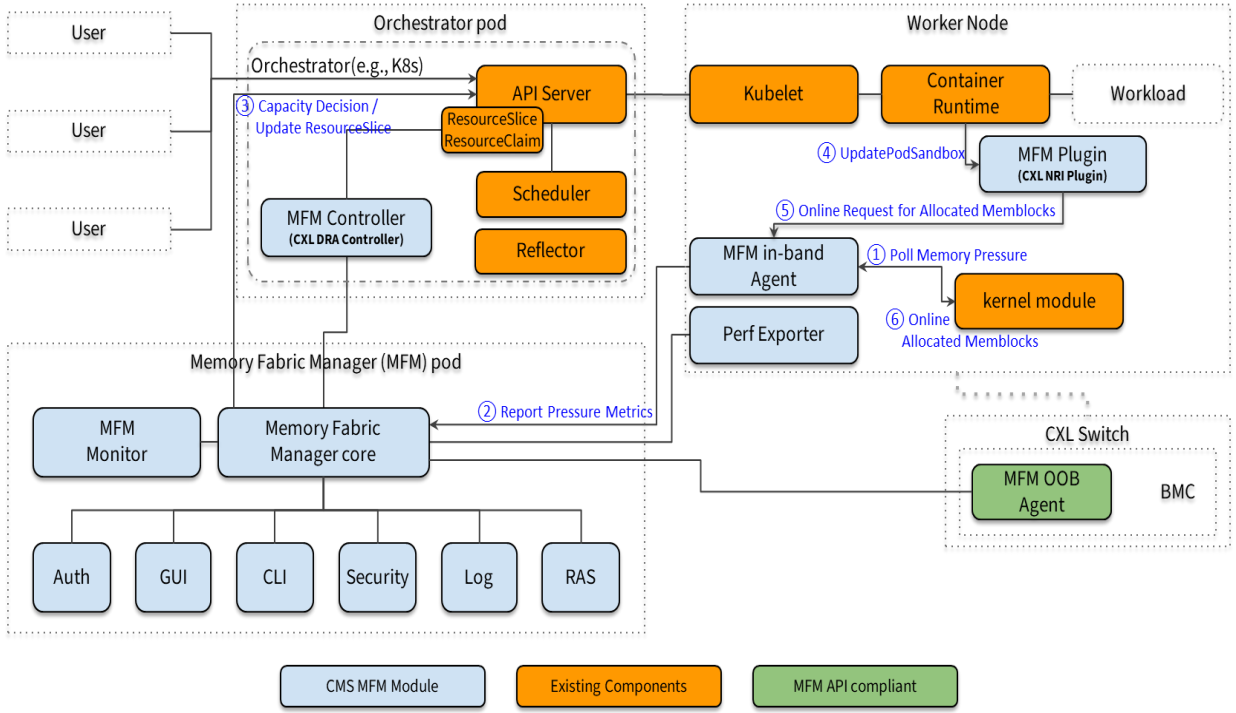
8.2

**SCENARIO**

A workload's memory demand fluctuates during execution — for example, an agentic AI framework whose tool invocations require a short-lived burst from 8 GiB to 32 GiB and back to 8 GiB. The pod's memory capacity is adjusted while it continues running, without a restart.

**KEY POINTS**

- Matches an agentic AI framework's characteristic behavior
  : bursty, generic working-set memory within one already-running pod.
- Uses Kubernetes' in-place Pod Resize mechanism
  (KEP-1287, GA since v1.35) and the cgroup memory.max path.
- No device-level semantics required
  : this is standard cgroup memory, not a device claim.



# Node-Level Memory Allocation via devdax

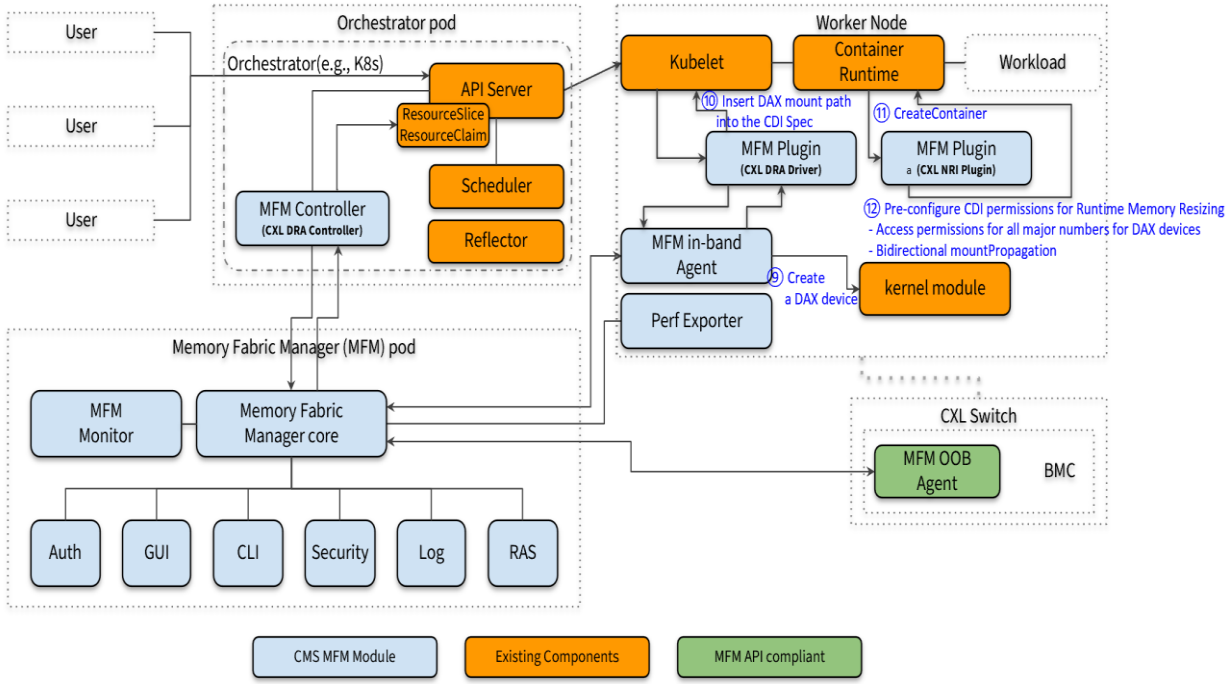
8.3

**SCENARIO**

A node hosts a memory-intensive service that manages its own memory placement directly through a device interface — for example, a CXL-enabled vector database such as Milvus. The composed memory is exposed as one or more devdax character devices of a pre-configured, fixed size per node, rather than as general-purpose System-RAM.

**KEY POINTS**

- Matches Milvus's behavior
  : a byte-addressable, memory-mapped region
  whose footprint is large and stable once deployed.
- Sized at node-provisioning time, not adjusted within a pod's runtime lifetime.
- Steps ①–⑧ reuse the same node-level DRA flow as Figure 8.1
  ; Figure 8.3 labels only the diverging steps, ⑨ onward.



# Pod-Level Dynamic devdax Unit Allocation

8.4

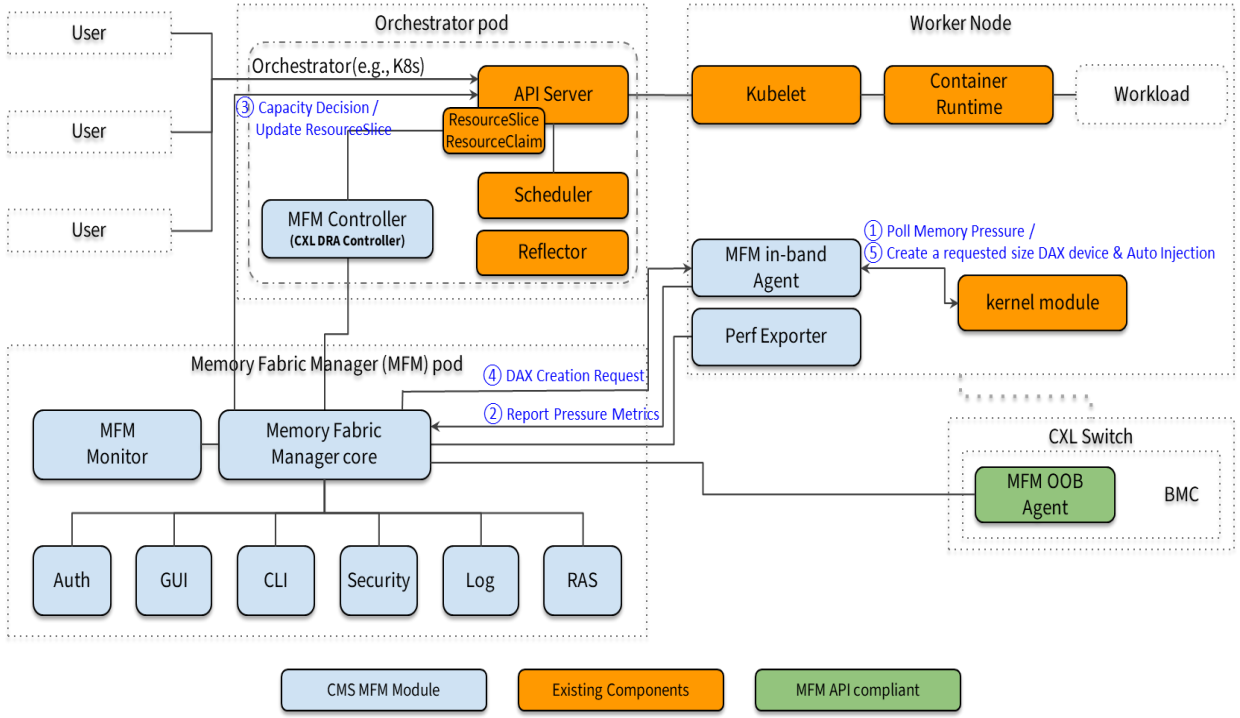
**SCENARIO**

A workload consumes composable memory through a byte-addressable device interface rather than System-RAM — for example, an LMCache-backed KV-cache tier using a CXL Logical Device / Dynamic Capacity Device. Capacity is added or removed as discrete devdax “dynamic units” (extents) while the pod continues running.

**KEY POINTS**

- Matches LMCache's behavior
  : it owns its KV-cache as a distinct, byte-addressable region to control placement and lifetime independently of ordinary heap accounting.
- The ResourceSlice update here is a post-hoc accounting step, not a scheduling gate — the pod is already running.

**Note —** Since no standard mechanism exists to re-apply a CDI Spec to a running container, everything needed is pre-configured at container-creation time ( §8.3, step ⑫). At runtime the new device is activated via “Auto Injection” — no Kubelet, NRI event, or API Server round-trip.



# [Call to Action]

- Problem to solve
  - Define the software for the memory fabric management layer
  - Software contribution, one module at a time
- Getting involved:
  - OCP wiki
    - https://www.opencompute.org/wiki/Server/CMS
    - https://www.opencompute.org/wiki/Server/CMS/DC-Memory-Fabric-Orchestration
  - Group.io
    - https://ocp-all.groups.io/g/OCP-CMS
    - https://ocp-all.groups.io/g/OCP-CMS-DC-Fabric-Orchestration
  - CMS weekly communication
    - Friday 9am PST Zoom call ( See detail in the wiki )
  - GitHub Contribution
    - Seagate - - CMA-CFM-Composability_Fabric_Manager
      - opencomputeproject/OCP-SVR-CMS-CFM-Composability_Fabric_Manager

# Thank You!

